

RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CP19211 – ADVANCE DATABASE TECHNOLOGIES

LABORATORY

Laboratory Record Note

Name	DEVARAJ A
Branch	ME-CSE
Register No	250711002
Semester	1
Academic Year	2025 -2027

**RAJALAKSHMI ENGINEERING
COLLEGE**
RAJALAKSHMI NAGAR, THANDALAM – 602 105

BONAFIDE CERTIFICATE

Name:DEVARAJ A

Academic Year : 2025 -2027

ODD Semester : I

Branch: ME-CSE

Register No.

250711002

Certified that this is the bonafide record of work done by the above student in the
Laboratory during the year

Signature of Faculty in-charge

Submitted for the Practical Examination held on

Internal Examiner

External Examiner

INDEX				
SNO	DATE	LIST OF EXPERIMENTS	PAGE NO.	SIGN
A	01/09/2025	Data Definition Language - Create, Alter and Drop - Enforce primary key, Foreign key, Check, Unique and Not Null Constraints - Creating views	04	
	8/09/2025	Data Manipulation Language - Insert, Delete, Update - Cartesian Product, Equi Join, Left outer join, Right outer join and full outer join - Aggregate Functions - Set operations - Nested Queries	09	
	15/09/2025	Transaction Control Language Commit, Rollback and Save points	29	
B	22/09/2025	Distributed Database Design and Implementation	33	
	29/09/2025	Row Level and Statement Level Triggers	37	
	06/10/2025	Accessing a Relational Database using Python	47	
C	13/10/2025	Creating XML Documents , Document type definition and XML Schema	51	
	27/10/2025	Using a Relational Database to store the XML documents as text	55	
	03/11/2025	Using a Relational Database to store the XML documents as Data elements	56	
	10/11/2025	Creating or Publishing customized XML Documents from Pre-existing Relational Databases	58	
	17/11/2025	Extracting XML Documents from Relational Databases	60	
	24/11/2025	XML Querying	63	
D	01/12/2025	- Creating Databases using MongoDB and - Writing simple queries to access databases created using Python	66	
E	08/12/2025	Implementing Access Control in Relational Databases	74	

EX.NO : A

DATE : 22 / 09 /2025

1. DATA DEFINITION LANGUAGES (DDL) COMMANDS OF BASE TABLES AND VIEWS

A Data Definition Language (DDL) Statement Is Used To Define The Database Structure Or Schema.

Aim:

To Study And Execute The DDL Commands In RDBMS.

DDL Commands:

- 1) CREATE
- 2) ALTER
- 3) RENAME
- 4) TRUNCATE
- 5) DROP

1) Create Table

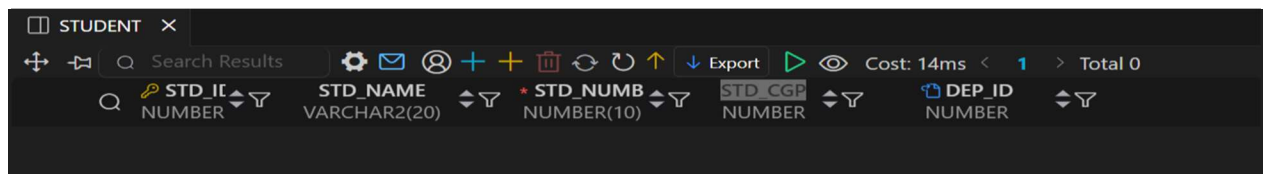
To Make A New Database, Table, Index, Or Stored Query. A Create Statement In SQL Creates An Object Inside Of A Relational Database Management System (RDBMS).

```
-- Create Table  
  
CREATE TABLE Player(  
    Id NUMBER PRIMARY KEY,  
    Name VARCHAR2(50),  
    Game VARCHAR2(50),  
    Location VARCHAR2(50)  
);
```

Output >>>>>>>>>>>>>>>

```
CREATE TABLE Player( id NUMBER PRIMARY KEY, name VARCHAR2(50), Game VARCHAR2(50), location VARCHAR2(50))  
  
AffectedRows : 0
```

[illegible]



5) DROP

Deletes a database object permanently.

```
-- Drop Table
DROP TABLE SPORTS;
```

Output >>>>>>>>>>>>>>>

5) Constraints

Department Table

```
-- Create Department + CONSTRAINT
CREATE TABLE Department (
    Dep_Id NUMBER PRIMARY KEY,    -- PRIMARY KEY
    Dep_Name VARCHAR2(20) UNIQUE -- UNIQUE
);
```

Output >>>>>>>>>>>>>>>

```
Result
+-----+
| CREATE TABLE Department ( dep_id NUMBER PRIMARY KEY, -- PRIMARY KEY dep_name VARCHAR2(20) UNIQUE UNIQUE ) |
+-----+
AffectedRows : 0
Cost: 962ms
```

Student Table

```
-- Create Student Table With FK + Constraint
```

```
CREATE TABLE Student (  
    Std_Id NUMBER PRIMARY KEY,           -- PRIMARY KEY  
    Std_Name VARCHAR2(20),  
    Std_Number NUMBER(10) NOT NULL UNIQUE, -- NOT NULL + UNIQUE  
    Std_Cgp NUMBER CONSTRAINT Chk_Cgp CHECK (Std_Cgp >= 7.5), -- CHECK  
    Dep_Id NUMBER,  
    CONSTRAINT Fk_Student FOREIGN KEY (Dep_Id)  
        REFERENCES Department(Dep_Id) ON DELETE SET NULL -- FOREIGN KEY + ON DELETE  
);  
  
SELECT * FROM STUDENT
```

Output >>>>>>>>>>>>

A screenshot of the SQL IDE toolbar. The 'Export' button is highlighted with a blue border. To its left is a search bar containing 'Search Results'. To its right are icons for undo, redo, and a green play button. Further right, the text 'Cost: 45ms' and 'Total 0' are visible. Below the toolbar, the column headers of the 'STUDENT' table are shown: 'STD_ID' (NUMBER), 'STD_NAME' (VARCHAR2(20)), 'STD_NUMBER' (NUMBER(10)), 'STD_CGP' (NUMBER), and 'DEP_IC' (NUMBER). Each header has a small icon to its right.

Teacher Table

```
-- Create Teacher
```

```
CREATE Table Teacher(  
    Teach_Id NUMBER PRIMARY KEY,  
    Teach_Name VARCHAR2(20) NOT NULL ,  
    Teach_Salary NUMBER(7,2),  
    Dep_Id NUMBER,  
    Constraint Fk_Teacher FOREIGN KEY(Dep_Id)  
        REFERENCES Department (Dep_Id) ON DELETE SET NULL);  
SELECT * FROM TEACHER;
```

Output >>>>>>>>>>>>>>>

TEACHER X

</

Result :

Thus executed the DDL commands in RDBMS

EX.NO:A

DATE : 29 / 09 /2025

2. DATA MANIPULATION LANGUAGE (DML) COMMANDS OF BASE TABLES AND VIEWS

Data manipulation language allows the users to query and manipulate data in existing schema in object. It allows following data to insert, delete, update and recovery data in schema object.

Aim:

To Study And Execute The DML Commands In RDBMS.

DDL Commands:

- INSERT
- UPDATE
- DELETE
- SELECT

1.) INSERT

Adds new records into a table.

1.1) Insert One Data

```
-- Insert One Data
INSERT INTO STUDENT (Id , Name , DEPARTMENT )
VALUES('1' , 'Deva' , 'Cse');
```

1.2) Insert Many Data

```
INSERT ALL
  INTO STUDENT (Id , Name , DEPARTMENT ) VALUES('2' , 'Jaya' , 'Bio' )
  INTO STUDENT (Id , Name , DEPARTMENT ) VALUES('3' , 'Pavi' , 'Mech' )
  INTO STUDENT (Id , Name , DEPARTMENT ) VALUES('4' , 'Boomika' , 'Civil' )
SELECT * FROM Dual;
```

Department Table

```
-- Create Department + CONSTRAINT
CREATE TABLE Department (
    Dep_Id NUMBER PRIMARY KEY,    -- PRIMARY KEY
    Dep_Name VARCHAR2(20) UNIQUE  -- UNIQUE
);
```

Insert Department Values

```
-- Insert Department Values
INSERT ALL
  INTO Department (Dep_Id, Dep_Name) VALUES (101, 'CSE')
  INTO Department (Dep_Id, Dep_Name) VALUES (102, 'MECH')
  INTO Department (Dep_Id, Dep_Name) VALUES (103, 'BIO')
  INTO Department (Dep_Id, Dep_Name) VALUES (104, 'CIVIL')
  INTO Department (Dep_Id, Dep_Name) VALUES (105, 'EEE')
  INTO Department (Dep_Id, Dep_Name) VALUES (106, 'ECE')
SELECT * FROM Dual;
SELECT * FROM Department;
```

Output >>>>>>>>>>>>>>>>>

```
Result X
+ + Search Results
Cost: 538ms < >

INSERT ALL
  INTO Department (dep_id, dep_name) VALUES (101, 'CSE')
  INTO Department (dep_id, dep_name) VALUES (102, 'MECH')
  INTO Department (dep_id, dep_name) VALUES (103, 'BIO')
  INTO Department (dep_id, dep_name) VALUES (104, 'CIVIL')
  INTO Department (dep_id, dep_name) VALUES (105, 'EEE')
  INTO Department (dep_id, dep_name) VALUES (106, 'ECE')
SELECT * FROM dual

AffectedRows : 6

DEPARTMENT X
+ + Search Results
Cost: 29ms < 1 > Total 6

DEP_ID DEP_NAME
NUMBER VARCHAR2(20)
> 101 CSE
> 102 MECH
> 103 BIO
> 104 CIVIL
> 105 EEE
> 106 ECE
```

Student Table

```
-- Create Student Table With FK + Constraint
CREATE TABLE Student (
  Std_Id NUMBER PRIMARY KEY,          -- PRIMARY KEY
  Std_Name VARCHAR2(20),
  Std_Number NUMBER(10) NOT NULL UNIQUE, -- NOT NULL + UNIQUE
  Std_Cgp NUMBER CONSTRAINT Chk_Cgp CHECK (Std_Cgp >= 7.5), -- CHECK
  Dep_Id NUMBER,
  CONSTRAINT Fk_Student FOREIGN KEY (Dep_Id)
    REFERENCES Department(Dep_Id) ON DELETE SET NULL -- FOREIGN KEY + ON DELETE
);
```

Insert Student Values

```
INSERT ALL
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (1, 'Gokul' ,
9585502916 , 7.5 ,101)
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (2, 'Sarath' ,
9585502917 , 8.5 ,102)
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (3, 'Deva' ,
9585502918 , 9.5 ,103)
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (4, 'Balaji' ,
9585502919 , 8.6 ,104)
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (5, 'Vicky' ,
9585502910 , 9.2 ,105)
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (6, 'Saranya' ,
9585502911 , 8.7 ,101)
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (7, 'Deva' ,
9585502912 , 7.8 ,102)
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (8, 'Boomika' ,
9585502913 , 8.9 ,103)
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (9, 'Gayathri' ,
9585502914 , 7.9 ,104)
  INTO STUDENT (STD_ID , STD_NAME , STD_NUMBER ,STD_CGP,DEP_ID) VALUES (10, 'Jaya' ,
9585502915 , 8.7 ,105)
SELECT * FROM Dual;
```

```
-- View STUDENT
SELECT * FROM STUDENT
```

```
SELECT * FROM STUDENT
```

Output >>>>>>>>>>>>>>

STUDENT

PropertiesDATALogERMonitor

SELECT * FROM STUDENT

Search Results

Export

Cost: 24ms < 1 > Total 10

	STD_ID NUMBER	STD_NAME VARCHAR2(20)	* STD_NUMB NUMBER(10)	STD_CGP NUMBER	DEP_ID NUMBER
>	1	gokul	9585502916	7.5	101
>	2	sarath	9585502917	8.5	102
>	3	deva	9585502918	9.5	103
>	4	balaji	9585502919	8.6	104
>	5	vicky	9585502910	9.2	105
>	6	saranya	9585502911	8.7	101
>	7	deva	9585502912	7.8	102
>	8	boomika	9585502913	8.9	103
>	9	gayathri	9585502914	7.9	104
>	10	jaya	9585502915	8.7	105

Teacher Table

```
-- Create Teacher
```

```
CREATE Table Teacher(
```

Teach Id NUMBER PRIMARY KEY,

Teach Name VARCHAR2(20) NOT NULL ,

Teach_Salary NUMBER(7,2),

Dep_Id NUMBER,

```
Constraint Fk_Teacher FOREIGN KEY(Dep_Id)
```

REFERENCES Department (Dep_Id) ON DELETE SET NULL

$$);$$

Insert Teacher Values

```
-- Insert Teacher
INSERT ALL
  INTO TEACHER (TEACH_ID , TEACH_NAME ,TEACH_SALARY ,DEP_ID) VALUES (1, 'Kannan' ,
25000,101)
  INTO TEACHER (TEACH_ID , TEACH_NAME ,TEACH_SALARY ,DEP_ID) VALUES (2, 'Praba' ,
15000,102)
  INTO TEACHER (TEACH_ID , TEACH_NAME ,TEACH_SALARY ,DEP_ID) VALUES (3, 'Siva' ,
20000,103)
  INTO TEACHER (TEACH_ID , TEACH_NAME ,TEACH_SALARY ,DEP_ID) VALUES (4, 'Suresh' ,18000
,104)
  INTO TEACHER (TEACH_ID , TEACH_NAME ,TEACH_SALARY ,DEP_ID) VALUES (5, 'Pavithra'
,32000,105)
INTO TEACHER (TEACH_ID , TEACH_NAME ,TEACH_SALARY ,DEP_ID) VALUES (6, 'Deva' ,32000,105)
SELECT * FROM Dual;
SELECT * FROM TEACHER;
```

Output >>>>>>>>>>>>>>>>

TEACHER				
TEACH_NUMBER	TEACH_NAME	TEACH_SALARY	TEACH_DEPT	
1	kannan	25000	101	
2	praba	15000	102	
3	siva	20000	103	
4	suresh	18000	104	
5	pavithra	32000	105	

2) Update Table

Modifies existing records in a table.

```
-- Update
UPDATE Student Set DEP_ID = 102
WHERE STD_NAME = 'Deva';

SELECT * FROM STUDENT
```

Output >>>>>>>>>>>>>>>

Search Results							Cost: 21ms < 1 > Total 1	
		STD_IL NUMBER	STD_NAME VARCHAR2(20)	* STD_NUMBER NUMBER(10)	STD_CGP NUMBER	DEP_ID NUMBER		
	>	1	gokul	9585502916	7.5	101		
	>	2	sarath	9585502917	8.5	102		
	>	3	deva	9585502918	9.5	102		
	>	4	balaji	9585502919	8.6	104		
	>	5	vicky	9585502910	9.2	105		
	>	6	saranya	9585502911	8.7	101		
	>	7	deva	9585502912	7.8	102		
	>	8	boomika	9585502913	8.9	103		

3) Delete Table

Removes specific records from a table.

```
-- Delete A Single Row
DELETE FROM Student
WHERE STD_ID = 2;
SELECT * FROM STUDENT
```

Output >>>>>>>>>>>>>>>>

STUDENT X

Search Results

Export Cost: 20ms

	STD_ID NUMBER	STD_NAME VARCHAR2(20)	* STD_NUMB NUMBER(10)	STD_CGP NUMBER	DEP_ID NUMBER
>	1	gokul	9585502916	7.5	101
>	3	deva	9585502918	9.5	102
>	4	balaji	9585502919	8.6	104

Output

BIO_DEPT X

Search Results

Export

Cost: 59ms < 1 > Total 2

*STD_ID NUMBER	STD_NAME VARCHAR2(20)	*STD_NUMB NUMBER(10)	STD_CGP NUMBER	DEP_ID NUMBER
> 3	deva	9585502918	9.5	103
> 8	boomika	9585502913	8.9	103

5) Foreign Key

A **Foreign Key** is a constraint that links one table to another by ensuring the values in a column match the primary key of another table.

Student With Department

-- Foreign Key (Student , Department)

```
SELECT S.Std_Name, D.Dep_Name
FROM Student S
JOIN Department D
    ON D.Dep_Id = S.Dep_Id;
```

Output >>>>>>>>>>>>>>>>

Teacher With Department

```
-- Foreign Key (Teacher , Department)
SELECT T.Teach_Name, D.Dep_Name
FROM TEACHER T
JOIN Department D
    ON D.Dep_Id = T.Dep_Id;
```

Output >>>>>>>>>>>>>>>

Student Teache Department

```
-- Foreign Key (Student , Teacher , Department) Using Join

SELECT
    S.STD_ID As Id,
    S.Std_Name As Student ,
    D.Dep_Name As Department,
    T.Teach_Name As Teacher
FROM Department D
JOIN Student S
    ON D.Dep_Id = S.Dep_Id
JOIN Teacher T
    ON D.Dep_Id = T.Dep_Id;
```

Output

ID NUMBER	STUDENT VARCHAR1	DEPARTMENT VARCHAR2	TEACHER VARCHAR1
> 1	gokul	CSE	kannan
> 2	sarath	MECH	praba
> 3	deva	BIO	siva
> 4	balaji	CIVIL	suresh
> 5	vicky	EEE	pavithra

6) AGGREGATE FUNCTIONS

Aggregate functions perform a calculation on a group of rows and return a single summary value (like total, average, count, min, or max).

6.1) COUNT() – Returns the number of rows.

```
-- Count
```

```
SELECT Count(Std_Id)As Total_Student FROM STUDENT ;
```

Output >>>>>>>>>>>>>>>>>>>>>>>>>

The screenshot shows a SQL IDE interface. At the top, there's a tab labeled "STUDENT". Below it, a toolbar contains various icons for search, settings, email, user management, undo, redo, and export. The main area displays the query results for the command "TOTAL_STUDENT". It shows a single row with the value "10". The status bar at the bottom indicates "Cost: 440ms" and "Total 1".

```
SELECT Count(Std_Id)As Total_Student FROM STUDENT ;
```

Output >>>>>>>>>>>>>>>>>

6.2) MAX() – Returns the largest value in a column.

[illegible]

```
SELECT Teach_Name, Teach_Salary
```

FROM Teacher

```
WHERE Teach_Salary = (SELECT MAX(Teach_Salary) FROM Teacher);
```

Output >>>>>>>>>>>>>>>



6.3) MIN() – Returns the smallest value in a column.

- Min

```
SELECT STD_ID , Std_Name, Std_Cgp
FROM STUDENT
WHERE STD_CGP = (SELECT Min(STD_CGP) FROM STUDENT);
```

Output >>>>>>>>>>>>>>>

6.4) SUM() – Returns the total of numeric values.

```
-- Sum
```

```
SELECT SUM(Teach_Salary) As Total_Salary FROM TEACHER;
```

Output >>>>>>>>>>>>>>>

The screenshot shows the top portion of a SQL IDE. The toolbar includes icons for undo, redo, search, and various query actions. The query results pane displays a single row with the column 'TOTAL_SALARY' and the value '110000'.

6.5) AVG() – Returns the average of numeric values.

- Avg

```
SELECT Avg(Teach_Salary) As Average_Salary FROM TEACHER;
```

Output >>>>>>>>>>>>>>>

The screenshot shows a SQL IDE interface. At the top, there's a tab labeled 'TEACHER' with a close button. Below the tab is a toolbar with various icons: a crosshair, a magnifying glass, a search bar containing 'Search Results', a gear icon, an envelope icon, a person icon, a plus icon, a minus icon, a trash icon, a refresh icon, a redo icon, an undo icon, a download icon, and an 'Export' button. To the right of the toolbar, it says 'Cost: 22ms' and '1' in a blue box, followed by 'Total 1'. Below the toolbar is a query editor with a search icon and the text 'AVERAGE_SALARY'. Below the query editor is a results pane with a search icon and the text '> 22000'.

7) **JOINS**

A **JOIN** is a SQL operation used to combine rows from two or more tables based on a related column between them.

7.1) INNER JOIN – Returns only the matching rows from both tables.

```
-- Inner JOIN  
  
SELECT  
    S.Std_Id AS Id,  
    S.Std_Name AS Name,  
    D.Dep_Name AS Department,  
    T.Teach_Name AS Teacher  
FROM Student S  
JOIN Department D  
    ON S.Dep_Id = D.Dep_Id  
JOIN Teacher T  
    ON T.Dep_Id = D.Dep_Id  
ORDER BY S.STD_ID ;  
  
Output >>>>>>>>>>>>>>>>
```

Result(RO) X

Search Results

Export

Cost: 269ms < 1 > Total 10

ID	NAME	DEPARTMENT	TEACHER
NUMBER	VARCHAR	VARCHAR2	VARCHAR
> 1	gokul	CSE	kannan
> 2	sarath	MECH	praba
> 3	deva	BIO	siva
> 4	balaji	CIVIL	suresh
> 5	vicky	EEE	pavithra
> 6	saranya	CSE	kannan

7.2) RIGHT JOIN (RIGHT OUTER JOIN) – Returns all rows from the right table and matching rows from the left table.

[illegible]

7.3) LEFT JOIN (LEFT OUTER JOIN) – Returns all rows from the left table and matching rows from the right table.

```
-- Left JOIN

SELECT
    S.Std_Id AS Id,
    S.Std_Name AS Name,
    D.Dep_Name AS Department,
    T.Teach_Name AS Teacher
FROM Student S
JOIN Department D
    ON S.Dep_Id = D.Dep_Id
RIGHT JOIN Teacher T
```

```
ON T.Dep_Id = D.Dep_Id
ORDER BY S.STD_ID;
```

Output >>>>>>>>>>>>>>>

Result(RO) X

Search Results

Export

Cost: 11ms < 1 > Total 10

ID	NAME	DEPARTMENT	TEACHER
NUMBER	VARCHAR	VARCHAR2	VARCHAR
> 1	gokul	CSE	kannan
> 2	sarath	MECH	praba
> 3	deva	BIO	siva
> 4	balaji	CIVIL	suresh
> 5	vicky	EEE	pavithra

7.4) FULL JOIN (FULL OUTER JOIN) – Returns all rows when there is a match in either table.

```
-- Full JOIN

SELECT
    S.Std_Id AS Id,
    S.Std_Name AS Name,
    D.Dep_Name AS Department,
    T.Teach_Name AS Teacher
FROM Student S
FULL JOIN Teacher T
    ON S.Dep_Id = T.Dep_Id
FULL JOIN Department D
    ON S.Dep_Id = D.Dep_Id OR T.Dep_Id = D.Dep_Id
WHERE S.Dep_Id = 101 OR S.Dep_Id IS NULL;
```

Output >>>>>>>>>>>>>>>

Result(RO) X

Search Results

Cost: 29ms < 1 > Total 3

ID	NAME	DEPARTMENT	TEACHER
NUMBER	VARCHAR	VARCHAR2	VARCHAR
> 1	gokul	CSE	kannan
> 6	saranya	CSE	kannan
> (NULL)	(NULL)	ECE	(NULL)

7.5) CROSS JOIN – Returns the Cartesian product of both tables (every row from table A with every row from table B).

```
-- CROSS Join

SELECT
    S.Std_Id AS Id,
    S.Std_Name AS Student_Name,
    D.Dep_Name AS Department,
    T.Teach_Name AS Teacher_Name
FROM Student S
JOIN Department D
    ON S.Dep_Id = D.Dep_Id
CROSS JOIN Teacher T
WHERE S.Dep_Id = 101
;
```

Output >>>>>>>>>>>>

Result(RO) X

Q Search Results

Export

Cost: 10ms < 1 > Total 10

ID NUMBER	STUDENT_NAME VARCHAR2	DEPARTMENT VARCHAR2	TEACHER_NAME VARCHAR2
> 1	gokul	CSE	kannan
> 1	gokul	CSE	praba
> 1	gokul	CSE	siva
> 1	gokul	CSE	suresh
> 1	gokul	CSE	pavithra
> 6	saranya	CSE	kannan
> 6	saranya	CSE	praba
> 6	saranya	CSE	siva
> 6	saranya	CSE	suresh
> 6	saranya	CSE	pavithra

8) AGREGATE FUNCTIONS + JOINS

Aggregate functions perform a calculation on a group of rows and return a single summary value (like total, average, count, min, or max).

```
-- Max
SELECT DEP_ID,Max(Teach_Salary) As Salary
FROM TEACHER
GROUP BY DEP_ID
;

-----

SELECT COUNT(*) AS Male_Count
FROM Student
WHERE Gender = 'Male' AND DEP_ID=101;

SELECT Teach_Id, Teach_Name, Teach_Salary
FROM Teacher
WHERE Teach_Salary = (SELECT MAX(Teach_Salary) FROM Teacher);

-----

SELECT S.Std_Id As Id ,Upper(S.Std_Name) As Name ,
       Upper(D.Dep_Name) As Department , S.STD_CGP As Cgp
FROM STUDENT S
JOIN DEPARTMENT D
On S.DEP_ID = D.DEP_ID
WHERE S.STD_CGP = (SELECT Max(STD_CGP) FROM STUDENT);

-----

SELECT
    S.Std_Id AS Id,
    UPPER(S.Std_Name) AS Name,
    UPPER(D.Dep_Name) AS Department,
    S.Std_Cgp AS Cgp,
    UPPER(T.Teach_Name) AS Teacher
FROM Student S
JOIN Department D
    ON S.Dep_Id = D.Dep_Id
LEFT JOIN Teacher T
    ON D.Dep_Id = T.Dep_Id
WHERE D.Dep_Id = 106
```

```
AND S.Std_Cgp = (
    SELECT Avg(Std_Cgp) FROM STUDENT
WHERE DEP_ID = 106
)
ORDER BY S.Std_Id;
```

9) SET OPERATIONS

Set operations combine the result sets of two SELECT queries and return a single result based on rules like **UNION**, **UNION ALL**, **INTERSECT**, and **MINUS**.

9.1) UNION – Combines results of two queries and removes duplicate rows.

```
-- Union
(SELECT Std_Name AS Student, NULL AS Teacher
FROM Student)
UNION
(SELECT NULL AS Student, Teach_Name AS Teacher
FROM Teacher);
```

Output >>>>>>>>>>>

Result(RO) X

Search Results

Export Cost: 14ms < 1 > Total 15

STUDENT VARCHAR	TEACHER VARCHAR
> balaji	(NULL)
> boomika	(NULL)
> deva	(NULL)
> gayathri	(NULL)
> gokul	(NULL)
> jaya	(NULL)
> saranya	(NULL)
> sarath	(NULL)
> vicky	(NULL)
> (NULL)	deva
> (NULL)	kannan
> (NULL)	pavithra
> (NULL)	praba
> (NULL)	siva
> (NULL)	suresh

9.2) UNION ALL – Combines results of two queries but keeps all duplicate rows.

```
-- Union All
(SELECT Std_Name AS Student, NULL AS Teacher
FROM Student)
UNION ALL
(SELECT NULL AS Student, Teach_Name AS Teacher
FROM Teacher);
```

Output >>>>>>>>>>>>>>>

9.3) INTERSECT – Returns only the rows that are common to both queries.

```
-- Intersect
SELECT Std_Name FROM Student
INTERSECT
SELECT Teach_Name FROM Teacher;
```

Output >>>>>>>>>>>>>

9.4) MINUS (*Oracle only*) – Returns rows from the first query that are not present in the second query.

```
-- Minus
(SELECT Std_Name AS Student, NULL AS Teacher
FROM Student)
Minus
(SELECT NULL AS Student, Teach_Name AS Teacher
FROM Teacher);
```

Output >>>>>>>>>>>>>

10) NESTED QUERIES

A **Nested Query** (or Subquery) is a query written inside another SQL query to provide results that the outer query uses.

```
-- Nested Queries
SELECT S.Std_Id As Id ,Upper(S.Std_Name) As Name ,
       Upper(D.Dep_Name) As Department , S.STD_CGP As Cgp
FROM STUDENT S
JOIN DEPARTMENT D
On S.DEP_ID = D.DEP_ID
WHERE S.STD_CGP = (SELECT Max(STD_CGP) FROM STUDENT);
```

Output >>>>>>>>>>>>>>>

Result(RO)

Search Results

ID NAME DEPARTMENT CGP
NUMBER VARCHAR2 VARCHAR2 NUMBER

> 3 DEVA BIO 9.5

RESULT :

Thus the data manipulation language (dml) of base tables and views are executed

EX.NO:A

DATE : 06 / 10 /2025

3. TRANSACTION CONTROL LANGUAGE

A Transaction Control Language (TCL) Statement Is Used To Define The Database Structure Or Schema.

Aim:

To Study And Execute The TCL Commands In RDBMS.

TCL Commands:

- COMMIT
- ROLLBACK
- SAVEPOINT

[illegible]

1) **COMMIT** – Saves all changes made in the current transaction permanently.

```
-- Insert first row
INSERT INTO Employee (name, age, salary)
VALUES ('John', 30, 50000);

-- Insert second row
INSERT INTO Employee (name, age, salary)
VALUES ('Meena', 28, 60000);
```

```
COMMIT;  
SELECT * FROM EMPLOYEE;
```

Output >>>>>>>>>>>>

EMPLOYEE X

Search Results

Cost: 467ms < 1 > Total 2

	NAME VARCHAR2(50)	AGE NUMBER	SALARY NUMBER
>	deva	30	50000
>	jaya	28	60000

2) ROLLBACK – Reverts all changes made in the current transaction.

```
-- Insert 3rd row
INSERT INTO Employee (name, age, salary)
VALUES ('pavi', 21, 20000);
```

```
SELECT * FROM EMPLOYEE;
```

Output >>>>>>>>>>>>>>>>

EMPLOYEE X

Search Results

Export

Cost: 13ms < 1 > Total 3

	NAME VARCHAR2(50)	AGE NUMBER	SALARY NUMBER
>	deva	30	50000
>	jaya	28	60000
>	pavi	21	20000

ROLLBACK;

```
SELECT * FROM EMPLOYEE;
```

Output >>>>>>>>>>>>>>>

EMPLOYEE X

Search Results

Export Cost: 467ms 1 Total 2

	NAME VARCHAR2(50)	AGE NUMBER	SALARY NUMBER
>	deva	30	50000
>	jaya	28	60000

3) SAVEPOINT – Creates a checkpoint inside a transaction to roll back to.

```
-- insert 3rd row
INSERT INTO Employee (name, age, salary)
VALUES ('gokul', 35, 70000);

SAVEPOINT sp1;

-- Insert 4h row
INSERT INTO Employee (name, age, salary)
VALUES ('boomika', 35, 70000);

-- Rollback to savepoint (undo boomika only)
ROLLBACK TO sp1;

-- Commit the final changes
COMMIT;

SELECT * FROM EMPLOYEE;
```

Output >>>>>>>>>>>>>>>

EMPLOYEE X

Search Results

Export Cost: 40ms < 1 > Total 3

	NAME VARCHAR2(50)	AGE NUMBER	SALARY NUMBER
>	deva	30	50000
>	jaya	28	60000
>	gokul	35	70000

RESULT:

Thus the Transaction Control Language (TCL) of base tables and views are executed.

EX.NO:B

DATE : 13 / 10 /2025

1) DISTRIBUTED DATABASE DESIGN AND IMPLEMENTATION

A **Distributed Database** is a database in which data is stored across multiple physical locations (computers or servers), but it appears to users as a single logical database.

Aim:

A **Distributed Database** is a database in which data is stored across multiple physical locations (computers or servers), but it appears to users as a single logical database.

Create Table India

```
-- student in india
CREATE TABLE STUDENT_INDIA (
    student_id    NUMBER PRIMARY KEY,
    name          VARCHAR2(100),
    age           NUMBER,
    course        VARCHAR2(50),
    country       VARCHAR2(50)
);

-- insert values
INSERT ALL
    INTO STUDENT_INDIA VALUES (1, 'deva', 21, 'B.Sc', 'India')
    INTO STUDENT_INDIA VALUES (2, 'jaya', 22, 'MBA', 'India')
    INTO STUDENT_INDIA VALUES (3, 'gokul', 20, 'B.Tech', 'India')
SELECT * FROM dual;

SELECT * FROM STUDENT_INDIA ;
```

Output >>>>>>>>>>>>>>>


```
CREATE DATABASE LINK siteB_link          -- private link particular person only
CONNECT TO deva IDENTIFIED BY deva2003
USING 'XE';

DROP DATABASE link siteC_link ;  -- drop link
```

Access Link

```
-- Access India
select * from student_india@siteA_link;
```

Output >>>>>>>>>>>

STUDENT_INDIA

Search Results

Export Cost: 19ms < 1 > Total 3

	STUDENT NUMBER	NAME VARCHAR2(100)	AGE NUMBER	COURSE VARCHAR2(50)	COUNTRY VARCHAR2(50)
>	1	deva	21	B.Sc	India
>	2	jaya	22	MBA	India
>	3	gokul	20	B.Tech	India

```
select * from student_japan@siteA_link;
```

Output >>>>>>>>>>>>>>>

STUDENT_JAPAN X

Search Results

Export Cost: 18ms < 1 > Total 3

	STUDENT_NUMBER	NAME	AGE	COURSE	COUNTRY
	VARCHAR2(100)	NUMBER	VARCHAR2(50)	VARCHAR2(50)	
>	101	Gojo	20	Engineering	Japan
>	102	luffy	22	Economics	Japan
>	103	Rei kamiki	21	Law	Japan

Join India with Japan

```
-- Join local STUDENT_INDIA with remote STUDENT_JAPAN
SELECT a.student_id, a.name AS india_student, b.name AS japan_student , a.age as
age
FROM STUDENT_INDIA a
Right JOIN STUDENT_JAPAN@siteB_link b
ON a.age = b.age;

select * from student_japan@siteA_link;
```

Output >>>>>>>>>>>>

STUDENT_ID	INDIA_STUDENT	JAPAN_STUDENT	AGE
NUMBER	VARCHAR2	VARCHAR2	NUMBER
> 1	deva	Rei kamiki	21
> 2	jaya	luffy	22
> 3	gokul	Gojo	20

RESULT:

Thus the base tables and views are executed.

EX.NO:B

DATE : 27 / 10 /2025

2. IMPLEMENTATION ROW LEVEL AND STATEMENT LEVEL TRIGGERS

A **Trigger** is a stored procedure in a database that automatically executes in response to certain events (INSERT, UPDATE, or DELETE) on a table or view.

Aim:

To implementation row level and statement level triggers in a database.

Types:

1. Row Level
2. Statement Level Triggers

1) Row Level Trigger

A trigger that fires **once for each row affected** by the triggering statement.

```
-- create table
CREATE TABLE WORKER (
    worker_id NUMBER PRIMARY KEY,
    name      VARCHAR2(100),
    age       NUMBER,
    work      VARCHAR2(50)
);

-- create page table
CREATE TABLE AUDIT_WORKER (
    audit_id    NUMBER PRIMARY KEY,
    worker_id   NUMBER,
    action      VARCHAR2(20),
    action_date DATE,
    old_name    VARCHAR2(100),
    new_name    VARCHAR2(100),
    old_age     NUMBER,
    new_age     NUMBER,
    old_work    VARCHAR2(50),
```

```

        new_work    VARCHAR2(50)
    );

-- Create Sequence
CREATE SEQUENCE audit_worker_seq START WITH 1 INCREMENT BY 1;

-- create table
CREATE TABLE WORKER (
    worker_id NUMBER PRIMARY KEY,
    name      VARCHAR2(100),
    age       NUMBER,
    work      VARCHAR2(50)
);

-- create page table
CREATE TABLE AUDIT_WORKER (
    audit_id    NUMBER PRIMARY KEY,
    worker_id   NUMBER,
    action      VARCHAR2(20),
    action_date DATE,
    old_name    VARCHAR2(100),
    new_name    VARCHAR2(100),
    old_age     NUMBER,
    new_age     NUMBER,
    old_work    VARCHAR2(50),
    new_work    VARCHAR2(50)
);

-- Create Sequence
CREATE SEQUENCE audit_worker_seq START WITH 1 INCREMENT BY 1;

-- Row level Trigger

-- Create Trigger
CREATE OR REPLACE TRIGGER trg_audit_worker
AFTER INSERT OR UPDATE OR DELETE ON WORKER
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO AUDIT_WORKER (
            audit_id, worker_id, action, action_date,
            old_name, new_name, old_age, new_age, old_work, new_work

```


INSERT ALL

```
SELECT * FROM WORKER; -- view worker table
```

WORKER

Search Results

Cost: 11ms < 1 > Total 3

	WORKER NUMBER	NAME VARCHAR2(100)	AGE NUMBER	WORK VARCHAR2(50)
>	1	deva	23	developer
>	2	jaya	27	designer
>	3	pavithra	23	Tester

Output >>>>>>>>>>>>>>>

```
-- delete row
```

Output >>>>>>>>>>>>>>>>>

WORKER X

Search Results

DELETE FROM WORKER WHERE NAME = 'pavithra'

AffectedRows : 1


```
SELECT * FROM WORKER; -- view worker table
```

Output >>>>>>>>>>>>

WORKER				
WORKER NUMBER	NAME	AGE	WORK	
1	deva	23	developer	
2	jaya	27	designer	

```
SELECT * FROM AUDIT_WORKER; -- view audit_worker table
```

Output >>>>>>>>>>>>>>>>>

AUDIT_WORKER									
AUDIT_NUMBER	WORKER_ID	ACTION	ACTION_DATE	OLD_NAME	NEW_NAME	OLD_AGE	NEW_AGE	OLD_WORK	NEW_WORK
NUMBER	NUMBER	VARCHAR2(20)	DATE	VARCHAR2(100)	VARCHAR2(100)	NUMBER	NUMBER	VARCHAR2(50)	VARCHAR2(50)
1	1	INSERT	2025-12-02 18:56:41	(NULL)	deva	(NULL)	23	(NULL)	developer
2	2	INSERT	2025-12-02 18:56:41	(NULL)	jaya	(NULL)	27	(NULL)	designer
3	3	INSERT	2025-12-02 18:56:41	(NULL)	pavithra	(NULL)	23	(NULL)	Tester
4	3	DELETE	2025-12-02 19:01:13	pavithra	(NULL)	23	(NULL)	Tester	(NULL)

```
-- Update row
```

```
UPDATE WORKER set work='hacker' WHERE NAME = 'deva';
```

```
SELECT * FROM WORKER; -- view worker table
```

Output >>>>>>>>>>>>>>>

WORKER				
	WORKER	NAME	AGE	WORK
	NUMBER	VARCHAR2(100)	NUMBER	VARCHAR2(50)
>	1	deva	23	hacker
>	2	jaya	27	designer

```
SELECT * FROM AUDIT_WORKER; -- view audit_worker table
```

Output >>>>>>>>>>>>>>>

AUDIT_NUMBER	WORKER_ID	ACTION	ACTION_DATE	OLD_NAME	NEW_NAME	OLD_AGE	NEW_AGE	OLD_WORK	NEW_WORK
1	1	INSERT	2025-12-02 18:56:41	(NULL)	deva	(NULL)	23	(NULL)	developer
2	2	INSERT	2025-12-02 18:56:41	(NULL)	jaya	(NULL)	27	(NULL)	designer
3	3	INSERT	2025-12-02 18:56:41	(NULL)	pavithra	(NULL)	23	(NULL)	Tester
4	3	DELETE	2025-12-02 19:01:13	pavithra	(NULL)	23	(NULL)	Tester	(NULL)
5	1	UPDATE	2025-12-02 19:07:12	deva	deva	23	23	developer	hacker
6	1	UPDATE	2025-12-02 19:07:29	deva	deva	23	23	hacker	developer
7	1	UPDATE	2025-12-02 19:07:35	deva	deva	23	23	developer	hacker

2 Statement Level Trigger

A trigger that fires **once per SQL statement**, regardless of how many rows are affected.

```
-- create table
CREATE TABLE WORKER (
    worker_id NUMBER PRIMARY KEY,
    name      VARCHAR2(100),
    age       NUMBER,
    work      VARCHAR2(50)
);

-- create page table
CREATE TABLE AUDIT_WORKER (
    audit_id    NUMBER PRIMARY KEY,
    worker_id   NUMBER,
    action      VARCHAR2(20),
    action_date DATE,
    old_name    VARCHAR2(100),
    new_name    VARCHAR2(100),
    old_age     NUMBER,
    new_age     NUMBER,
    old_work    VARCHAR2(50),
    new_work    VARCHAR2(50)
);

-- Create Sequence
CREATE SEQUENCE audit_worker_seq START WITH 1 INCREMENT BY 1;

-- create table
CREATE TABLE WORKER (
    worker_id NUMBER PRIMARY KEY,
    name      VARCHAR2(100),
    age       NUMBER,
```

```

        work          VARCHAR2(50)
    );

-- create page table
CREATE TABLE AUDIT_WORKER (
    audit_id          NUMBER PRIMARY KEY,
    worker_id         NUMBER,
    action            VARCHAR2(20),
    action_date       DATE,
    old_name          VARCHAR2(100),
    new_name          VARCHAR2(100),
    old_age           NUMBER,
    new_age           NUMBER,
    old_work          VARCHAR2(50),
    new_work          VARCHAR2(50)
);

-- Create Sequence
CREATE SEQUENCE audit_worker_seq START WITH 1 INCREMENT BY 1;

-- Statement level Trigger

-- Create Trigger
CREATE OR REPLACE TRIGGER trg_audit_worker_stmt
AFTER INSERT OR UPDATE OR DELETE ON WORKER
BEGIN
    INSERT INTO AUDIT_WORKER (
        audit_id, worker_id, action, action_date,
        old_name, new_name, old_age, new_age, old_work, new_work
    )
    VALUES (
        audit_worker_seq.NEXTVAL,
        NULL,
        CASE
            WHEN INSERTING THEN 'INSERT'
            WHEN UPDATING THEN 'UPDATE'
            WHEN DELETING THEN 'DELETE'
        END,
        SYSDATE,
        NULL, NULL, NULL, NULL, NULL, NULL
    );
END;
```

```
Script Output x
Task completed in 0.381 seconds

Trigger TRG_WORKER_AUDIT compiled
```

INSERT ALL

```
SELECT * FROM WORKER; -- view worker table
```

WORKER

Search Results

Export Cost: 11ms < 1 > Total 3

	WORKER NUMBER	NAME VARCHAR2(100)	AGE NUMBER	WORK VARCHAR2(50)
>	1	deva	23	developer
>	2	jaya	27	designer
>	3	pavithra	23	Tester

AUDIT_WORKER										
Search Results										
Cost: 90ms < 1 > Total 3										
	AUDIT_NUMBER	WORKER_IC_NUMBER	ACTION VARCHAR2(20)	ACTION_DATE DATE	OLD_NAME VARCHAR2(100)	NEW_NAME VARCHAR2(100)	OLD_AGE_NUMBER	NEW_AG_NUMBER	OLD_WORK VARCHAR2(50)	NEW_WO VARCHAR2(50)
>	1	1	INSERT	2025-12-02 18:56:41	(NULL)	deva	(NULL)	23	(NULL)	developer
>	2	2	INSERT	2025-12-02 18:56:41	(NULL)	jaya	(NULL)	27	(NULL)	designer
>	3	3	INSERT	2025-12-02 18:56:41	(NULL)	pavithra	(NULL)	23	(NULL)	Tester

```
DELETE FROM WORKER WHERE NAME = 'pavithra';
```


Output

AUDIT NUMBER	WORKER_ID NUMBER	ACTION VARCHAR2(20)	ACTION_DATE DATE	OLD_NAME VARCHAR2(100)	NEW_NAME VARCHAR2(100)	OLD_AG NUMBER	NEW_AG NUMBER	OLD_WORK VARCHAR2(50)	NEW_WORK VARCHAR2(50)
1	1	INSERT	2025-12-02 18:56:41	(NULL)	deva	(NULL)	23	(NULL)	developer
2	2	INSERT	2025-12-02 18:56:41	(NULL)	jaya	(NULL)	27	(NULL)	designer
3	3	INSERT	2025-12-02 18:56:41	(NULL)	pavithra	(NULL)	23	(NULL)	Tester
4	3	DELETE	2025-12-02 19:01:13	pavithra	(NULL)	23	(NULL)	Tester	(NULL)
5	1	UPDATE	2025-12-02 19:07:12	deva	deva	23	23	developer	hacker
6	1	UPDATE	2025-12-02 19:07:29	deva	deva	23	23	hacker	developer
7	1	UPDATE	2025-12-02 19:07:35	deva	deva	23	23	developer	hacker

RESULT:

Thus the tables and views are executed.

Create Table

```
# create Table
oracle.execute('''
create table player(
            id number ,
            name varchar2(20),
            game varchar2(20)
        )
''')
print('table created')
```

```
Output >>>>>>>>>>>  
table created
```

Insert Row

```
#insert

oracle.execute('''
insert all
            into player values(1, 'deva' , 'kabaddi')
            into player values(2, 'jaya' , 'chess')
select * from dual

''')

connect.commit
print('row inserted')
```

```
Output >>>>>>>>>>>  
row inserted
```

View Table

```
# display
oracle.execute('select * from player')
row = oracle.fetchall()
for i in row:
    print(i)
```

[illegible]

Result:

Thus To Accessing a Relational Database using Python . Has been completed successfully

EX.NO : C

DATE : 13 / 10 /2025

1) CREATING XML DOCUMENTS, DOCUMENT TYPE DEFINITION AND XML SCHEMA

Aim:

To create an XML document, define its structure using DTD, and validate it using XML Schema.

Procedure:

- 1) Create an XML Document: Write an XML document using a text editor and save it with the xml extension.
- 2) Define DTD: Define the document structure in a .dtd file.
- 3) Validate with XML Schema (XSD): Create an XML Schema file (.xsd) to validate the XML document.

Create XML

Developers.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- developers.xml - simple element-based structure -->
<developers>

    <developer>
        <id>1</id>
        <name>deva</name>
        <role>ML Developer</role>
        <skills>python</skills>
    </developer>

    <developer>
        <id>2</id>
        <name>jaya</name>
        <role>Frontend Developer</role>
        <skills>React</skills>
    </developer>
</developers>
```

Create XML With DTD

Developers.dtd

```
<?xml version="1.0" encoding="UTF-8"?>

<!ELEMENT developers (developer+)>

<!ELEMENT developer (id, name, role, skills)>
<!ELEMENT id (#PCDATA)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT role (#PCDATA)>
<!ELEMENT skills (#PCDATA)>
```

Connect dtd in Developers.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE developers SYSTEM "developers.dtd"> <!-- developers.xtd -->

<developers>

    <developer>
        <id>1</id>
        <name>deva</name>
        <role>ML Developer</role>
        <skills>python</skills>
    </developer>

    <developer>
        <id>2</id>
        <name>jaya</name>
        <role>Frontend Developer</role>
        <skills>React</skills>
    </developer>

</developers>
```

Create Schema

Developers.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Root element -->
  <xs:element name="developers">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="developer" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="id" type="xs:integer"/>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="role" type="xs:string"/>
              <xs:element name="skills" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Developers.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<developers xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="developers.xsd"> <!--developers.xsd -->

  <developer>
    <id>1</id>
    <name>deva</name>
    <role>ML Developer</role>
    <skills>python</skills>
  </developer>
```

```
<developer>
  <id>2</id>
  <name>jaya</name>
  <role>Frontend Developer</role>
  <skills>React</skills>
</developer>

</developers>
```

RESULT:

The XML document is structured and validated using DTD & XML.

EX.NO : C

DATE : 27 / 10 /2025

2) STORE THE XML DOCUMENTS AS TEXT IN ORACLE

Aim:

To store an XML document as text in OracleSQL.

Procedure:

- 1) Create a table to store the XML document as text.
- 2) Insert the XML document into the table.

Create Table

```
-- create xml Table
CREATE TABLE developer_xml (
    id NUMBER,
    xml_data XMLTYPE
);

SELECT * FROM DEVELOPER_XML;
```

Output >>>>>>>>>>>>>

A screenshot of the SQL IDE toolbar. It includes icons for file management (new, open, save, close), search (magnifying glass), settings (gear), and execution (play, stop). The 'Export' button is highlighted with a green border. The toolbar also displays 'Cost: 42ms' and 'Total 0'.

Insert XML

```
INSERT INTO developer_xml (id, xml_data)
VALUES ( 1,
        XMLTYPE('
            <developers>
              <developer>
                <id>1</id>
                <name>deva</name>
                <role>ML Developer</role>
```


EX.NO: C

DATE : 03 / 11 /2025

3) STORE THE XML DOCUMENTS AS DATA ELEMENTS IN ORACLE

Aim:

To store an XML document as individual elements in OracleSQL.

Procedure:

- 1) Create a table with columns for individual elements.
- 2) Parse the XML and insert its elements as rows in the table.

View XML Table Format

```
SELECT x.id, x.name, x.role, x.skills
FROM developer_xml d,
XMLTable( '/developers/developer'
PASSING d.xml_data COLUMNS
        id      NUMBER          PATH 'id',
        name    VARCHAR2(50)    PATH 'name',
        role    VARCHAR2(50)    PATH 'role',
        skills  VARCHAR2(200)   PATH 'skills'
) x ;
```

Output >>>>>>>>>>>>>>>

[illegible]

RESULT:

XML elements are successfully stored as relational data.

EX.NO:C

DATE : 10 / 11 /2025

4.) PUBLISHING A CUSTOMIZED XML DOCUMENT FROM RELATIONAL DATA

Aim:

To create a customized XML document from relational data.

Procedure:

- 1) Use the `xmlelement()` function to generate an XML document.

Create table

```
-- Create table
```

```
CREATE Table akatsuki(  
    id NUMBER PRIMARY key,  
    name VARCHAR2(20) ,  
    role VARCHAR2(20),  
    score NUMBER  
);
```

Output >>>>>>>>>>>>>>

Result X

+ - 🔍 Search Results ⚙️ 📧 👤 + ✖ ↺ ↻ ↑ ↓ Export ▶ 👁 Cost: 579ms < >

```
CREATE Table akatsuki( id NUMBER PRIMARY KEY, name VARCHAR2(20) , role VARCHAR2(20), score NUMBER )
```

AffectedRows : 0

Insert data

```
-- insert data
INSERT ALL
  INTO AKATSUKI(id , name , role , score) VALUES(1 , 'deva' , 'pain' , 100)
  INTO AKATSUKI(id , name , role , score) VALUES(2 , 'jaya' , 'topy' , 100)
  INTO AKATSUKI(id , name , role , score) VALUES(3 , 'boomika' , 'konan' , 100)
  INTO AKATSUKI(id , name , role , score) VALUES(4 , 'pavi' , 'itachi' , 100)
```

[illegible]

Publish XML

[illegible]

RESULT:

Relational data is successfully converted to a structured XML document.

EX.NO:C

DATE : 17 / 11 /2025

5) EXTRACTING XML DOCUMENTS FROM A RELATIONAL DATABASE

Aim:

To extract stored XML documents from OracleSQL.

Procedure:

- 1) Retrieve the stored XML data using SQL queries.

Create table

```
-- create xml Table
CREATE TABLE developer_xml (
    id NUMBER,
    xml_data XMLTYPE
);

SELECT * FROM DEVELOPER_XML;
```

Output >>>>>>>>>>>>>

Insert XML

```
INSERT INTO developer_xml (id, xml_data)
VALUES ( 1,
        XMLTYPE('
            <developers>

                <developer>
                    <id>1</id>
                    <name>deva</name>
                    <role>ML Developer</role>
                    <skills>python</skills>
                </developer>
            </developers>
        '))
```

Output >>>>>>>>>>>>

> 1 <developers> Text Load Export Premium Only

> 1 <developers> Text Load Export Premium Only

Extrat XML

Output >>>>>>>>>>>>>>>

DEVELOPER_XML

Search Results [Settings] [Email] [User Icon] [+][+][Trash Icon][Refresh Icon][Undo Icon][Up Arrow] Export Cost: 35ms < 1 > Total 2

ID NUMBER	NAME	ROLE	SKILLS
> 1	deva	ML Developer	python
> 2	jaya	Frontend Developer	React

RESULT:

Stored XML document extracted successfully.

EX.NO:C

DATE : 24 / 11 /2025

6) QUERYING XML DATA

Aim:

To perform XMLTable -based queries on XML data in OracleSQL.

Procedure:

- 1) Use the `XMLTable` function to query nodes in the XML document.

Create Table

```
-- create xml Table
CREATE TABLE developer_xml (
    id NUMBER,
    xml_data XMLTYPE
);

SELECT * FROM DEVELOPER_XML;
```

Output >>>>>>>>>>>>>>>>>

Insert XML

```
INSERT INTO developer_xml (id, xml_data)
VALUES ( 1,
        XMLTYPE('
          <developers>

            <developer>
              <id>1</id>
              <name>deva</name>
              <role>ML Developer</role>
            </developer>
          </developers>
        '))
```

Output >>>>>>>>>>>>

> 1 <developers> ↓ < – Text – Load Export Premium Only

```
SELECT x.id, x.name, x.role, x.skills
FROM developer_xml d,
XMLTable( '/developers/developer'
PASSING d.xml_data COLUMNS
        id      NUMBER          PATH 'id',
        name    VARCHAR2(50)    PATH 'name',
        role    VARCHAR2(50)    PATH 'role',
        skills  VARCHAR2(200)   PATH 'skills'
) x ;
```

Output >>>>>>>>>>>>>

DEVELOPER_XML X				
Search Results				
Cost: 35ms < 1 > Total 2				
	ID NUMBER	NAME	ROLE	SKILLS
>	1	deva	ML Developer	python
>	2	jaya	Frontend Developer	React

RESULT:

The XML document is successfully queried using XPath.

EX.NO:D

DATE : 01 / 12 /2025

1) CREATING DATABASES USING MONGODB IN PYTHON

Aim:

To perform XMLTable -based queries on XML data in OracleSQL.

Procedure:

Install PIP

```
pip install pymongo
```

Connect DataBase

```
from pymongo import MongoClient

# 1. Connect to MongoDB (local)
client = MongoClient("mongodb://localhost:27017/")

print("Connection created successfully!")
```

Output >>>>>>>>>>

```
Connection created successfully!
```

Create DataBase

```
# 2. Create / select database
db = client["coe_red"] # database name

print(f"DataBase created successfully!")
```

[illegible]

Create Collection

```
# 3. Create / select collection
developer = db["developer"]    # Collection name

print("Collection created successfully!")
```

Output >>>>>>>>>>>>>

Collection created successfully!

Insert Many Collection

```
# insert Collection
developer1 = {
    "id": 1,
    "name": "Deva",
    "role": "ML Developer",
    "skills": ["Python", "TensorFlow"]
}

developer2 = {
    "id": 2,
    "name": "Jaya",
    "role": "Frontend Developer",
    "skills": ["React", "JavaScript"]
}

developer.insert_many([developer1, developer2])

print("Collection inserted!")
```

Output >>>>>>>>>>>>>>>>>

Collection inserted!

Insert One Collection

```
# insert Collection

# ONE collection
developer3 = {
    "id": 3,
    "name": "pavithra",
    "role": " java Developer",
    "skills": ["java", "Springboot"]}
}

developer.insert_one(developer3)

print("Collection inserted!")
```

```
Output >>>>>>>>>>  
Collection inserted!
```

Find Collection

```
# view collection
result = developer.find()

for data in result:
    print(data)
```

Output >>>>>>>>>>>>>>

```
{'_id': ObjectId('692fef6c9b8d336ea95359f3'), 'id': 1, 'name': 'Deva', 'role': 'ML Developer', 'skills': ['Python', 'TensorFlow']}
```

```
{'_id': ObjectId('692fef6c9b8d336ea95359f4'), 'id': 2, 'name': 'Jaya', 'role': 'Frontend Developer', 'skills': ['React', 'JavaScript']}
```

```
{'_id': ObjectId('693015329b8d336ea95359f9'), 'id': 3, 'name': 'pavithra', 'role': 'java Developer', 'skills': ['java', 'Springboot']}
```

```
# find deva
result = developer.find({"name": 'Deva'})

for data in result:
    print(data)

Output >>>>>>>>>>>>>>

{'_id': ObjectId('692fef6c9b8d336ea95359f3'), 'id': 1, 'name': 'Deva', 'role':
'ML Developer', 'skills': ['Python', 'TensorFlow']}


# find java

result = developer.find({"skills":'java'})

for data in result:
    print(data)

Output >>>>>>>>>>>>>>

{'_id': ObjectId('693015329b8d336ea95359f9'), 'id': 3, 'name': 'pavithra',
'reole': ' java Developer', 'skills': ['java', 'Springboot']}
```

```
{'_id': ObjectId('692fef6c9b8d336ea95359f3'), 'id': 1, 'name': 'Deva', 'role': 'ML Developer', 'skills': ['Python', 'TensorFlow']}
```

```
Output >>>>>>>>>>>>>>>>
{'_id': ObjectId('693015329b8d336ea95359f9'), 'id': 3, 'name': 'pavithra',
'role': ' java Developer', 'skills': ['java', 'Springboot']}
```

```
{'_id': ObjectId('693015329b8d336ea95359f9'), 'id': 3, 'name': 'pavithra',  
'role': ' java Developer', 'skills': ['java', 'Springboot']}
```

```
# Update one record
developer.update_one(
    {"name": "deva"},
    {"$set": {"role": "Senior Python Developer"}} # Update new data
)

print('Uptada Successfully! ')
```

Update Successfully!

Output >>>>>>>>>>>>>

```
# Update one record
developer.update_one(
    {"name": "pavithra"},
    {"$pull": {"skills": "java"}} # remove java
)
```

Output >>>>>>>>>>>>>>>

```
# view collection
result = developer.find({"name": 'pavithra'})
```

Output >>>>>>>>>>>>>>>

```
# Update one record
developer.update_one(
    {"name": "Deva"},
    {"$unset": {"skills": ""}} # delete Skills
)
```

Output >>>>>>>>>>>>>>

```
# view collection
result = developer.find({"name": 'Deva'})
```

```
for data in result:
    print(data)
```

```
{'_id': ObjectId('693014219b8d336ea95359f7'), 'id': 1, 'name': 'Deva', 'role': 'Senior Python Developer'}
```

Output >>>>>>>>>>>>>>>

```
# view collection
result = developer.find()

for data in result:
    print(data)
```

Output >>>>>>>>>>>>>>

```
{'_id': ObjectId('693015329b8d336ea95359f9'), 'id': 3, 'name': 'pavithra',  
'role': ' java Developer', 'skills': ['Springboot'], 'company': 'TCS'}
```

Delete

```
# one record
developer.delete_one({"name": 'Jaya'})

print('delete Successfully!')
```

Output >>>>>>>>>>>>>

```
delete Successfully!
```

```
# view collection
result = developer.find()

for data in result:
    print(data)
```

Output >>>>>>>>>>>>>

```
{'_id': ObjectId('693014219b8d336ea95359f7'), 'id': 1, 'name': 'Deva', 'role': 'Senior Python Developer', 'company': 'TCS'}
```

```
{'_id': ObjectId('693015329b8d336ea95359f9'), 'id': 3, 'name': 'pavithra',  
'role': ' java Developer', 'skills': ['Springboot'], 'company': 'TCS'}
```

```
# many record
developer.delete_many({"company": "TCS"})

print('delete succesfully! ')
```

Output >>>>>>>>>>>>>>>

```
delete Successfully!
```

```
# Delete Entire collection data
developer.delete_many({})

print('all collections delete successfully! ')
```


Output >>>>>>>>>>>>>

```
all collection delete successfully!
```

Drop

Drop Collection

```
developer.drop()
```

```
print('delete collection sucessfully!')
```

Output >>>>>>>>>>>>

```
delete collection successfully!
```

Drop Collection

```
client.drop_database("code_red")
```

```
print('delete database sucessfully!')
```

Output >>>>>>>>>>>>

```
delete database successfully!
```

RESULT:

Thus to Create Databases using MongoDB and write simple queries to access databases using Python has been completed successfully.

EX.NO:E

DATE : 08 / 12 /2025

IMPLEMENTING ACCESS CONTROL IN RELATIONAL DATABASE

Aim:

To Implement access control in relational databases is crucial for ensuring the security and integrity of data.

Procedure:

- 1) **Download** the latest Oracle SQL Server version from the official Microsoft website.
- 2) **Create Roles:** Define a roles for storing customer data.
- 3) **Create Users:** Define users with login credentials.
- 4) **Grant Permissions:** Assign access permissions to the users for the table.
- 5) **Policy Enforcement:** The filter ensures that each user can only see or modify their own data based on their name.

Create Table

```
-- Create Table

CREATE Table bank(
    id NUMBER PRIMARY KEY,
    name VARCHAR2(20),
    role VARCHAR2(20)
);
```

```
-- -----Create Admin Role-----  
  
CREATE ROLE Admin_Role; -- Overall Access  
  
Output >>>>>>>>>>>>  
2025-12-03 18:28:52 [ INFO] Executing: CREATE ROLE Admin_Role  
2025-12-03 18:28:52 [ INFO] Result: 0 rows affected in 13ms  
  
-- assign privileges to role  
GRANT CREATE TABLE, ALTER ANY TABLE TO Admin_Role;  
  
Output >>>>>>>>>>>>  
2025-12-03 18:29:01 [ INFO] Executing: GRANT CREATE TABLE, ALTER ANY TABLE TO  
Admin_Role  
2025-12-03 18:29:01 [ INFO] Result: 0 rows affected in 16ms  
  
-- Create User  
CREATE USER luffy IDENTIFIED BY luffy123;  
  
Output >>>>>>>>>>>>  
2025-12-03 18:29:41 [ INFO] Executing: CREATE USER luffy IDENTIFIED BY luffy123  
2025-12-03 18:29:42 [ INFO] Result: 0 rows affected in 624ms  
  
-- Assign role to user  
GRANT Admin_Role TO luffy;  
  
Output >>>>>>>>>>>>  
2025-12-03 18:30:18 [ INFO] Executing: GRANT Admin_Role TO luffy  
2025-12-03 18:30:18 [ INFO] Result: 0 rows affected in 23ms  
  
-- ----- Create Manager Role-----  
  
CREATE ROLE Manager_Role; -- Write & Read Access  
  
Output >>>>>>>>>>>>
```

```
2025-12-03 18:31:53 [ INFO] Executing: CREATE ROLE Manager_Role
2025-12-03 18:31:53 [ INFO] Result: 0 rows affected in 13ms
```

```
-- assign privileges to role
GRANT SELECT, INSERT, UPDATE ON deva.bank TO Manager_Role;
```

Output >>>>>>>>>>>>>

```
2025-12-03 18:42:31 [ INFO] Executing: GRANT SELECT, INSERT, UPDATE ON deva.bank
TO Manager_Role
2025-12-03 18:42:31 [ INFO] Result: 0 rows affected in 35ms
```

```
-- Create User
CREATE USER jaya IDENTIFIED BY jaya123;
```

Output >>>>>>>>>>>>>>>

```
2025-12-03 18:42:47 [ INFO] Executing: CREATE USER jaya IDENTIFIED BY jaya123
2025-12-03 18:42:47 [ INFO] Result: 0 rows affected in 28ms
```

```
-- Assign role to user
GRANT Manager_Role TO jaya;
```

Output >>>>>>>>>>>>>>>

```
2025-12-03 18:42:50 [ INFO] Executing: GRANT Manager_Role TO jaya
2025-12-03 18:42:50 [ INFO] Result: 0 rows affected in 16ms
```

```
-- ----- Create Staff Role-----
```

```
CREATE Role Staff_Role; -- Read Access
```

Output >>>>>>>>>>>>>>

```
2025-12-03 18:42:56 [ INFO] Executing: CREATE Role Staff_Role
2025-12-03 18:42:50 [ INFO] Result: 0 rows affected in 16ms
```

```
-- assign privileges to role
GRANT SELECT ON deva.bank TO Staff_Role;
```

Output >>>>>>>>>>>>>>

```
2025-12-03 18:43:11 [ INFO] Executing: GRANT SELECT ON deva.bank TO Staff_Role
2025-12-03 18:43:11 [ INFO] Result: 0 rows affected in 15ms
```

```
CREATE USER boomika IDENTIFIED BY boomika123;
```

```
2025-12-03 18:43:14 [ INFO] Executing: CREATE USER boomika IDENTIFIED BY boomika123
```

```
-- Assign role to user
```

```
2025-12-03 18:43:16 [ INFO] Executing: GRANT Staff_Role TO boomika
```

```
ALTER Table BANK add salary NUMBER DEFAULT '1000';
```

```
SELECT * FROM BANK;
```

Output >>>>>>>>>>>

BANK

Search Results Export Cost: 45ms < 1 > Total 1

ID NUMBER	NAME VARCHAR2(20)	ROLE VARCHAR2(20)	SALARY NUMBER
> 1	deva	manager	1000

```
-- alter table del co;
ALTER Table BANK DROP COLUMN salary;
```

```
SELECT * FROM BANK;
```

Output >>>>>>>>>>>

Manager Login

```
-- Switch user
GRANT DBA TO JAYA;
```

```
-- Insert row
```

```
INSERT INTO bank (id , name , role) VALUES(2 , 'pavi' , 'staff');
```

```
SELECT * FROM bank;
```

Output >>>>>>>>>>>>>

The screenshot shows a SQL IDE window with a table named 'bank'. The table has three columns: 'ID' (NUMBER), 'NAME' (VARCHAR), and 'ROLE' (VARCHAR). The table contains two rows of data: (1, deva, manager) and (2, pavi, staff). The interface includes a search bar at the top, a table view icon, and a 'Cost: 16ms' indicator.

```
-- Delete row
```

```
SELECT * FROM bank;
```

Output >>>>>>>>>>>>>>>

A screenshot of a SQL editor interface. At the top, there's a tab labeled 'bank'. Below the tab is a search bar with the text 'Search Results'. To the right of the search bar is a toolbar with various icons: a gear, an envelope, a person, a plus sign, a trash can, a refresh icon, a circular arrow, an up arrow, a down arrow, and an 'Export' button. Further right, there's a status bar showing 'Cost: 16ms' and 'Total 1'. Below the toolbar is a table with three columns: 'ID', 'NAME', and 'ROLE'. The 'ID' column has a sub-header 'NUMBER'. The 'NAME' column has a sub-header 'VARCHAR'. The 'ROLE' column has a sub-header 'VARCHAR'. The table has one row with the values '1', 'deva', and 'manager'.

```
-- alter table del co;
```

```
ALTER Table BANK DROP COLUMN salary;
```

Output >>>>>>>>>>>>

```
2025-12-03 19:35:44 [ERROR] ORA-00904: "SALARY": invalid identifier
```

Delete

```
-- Drop Privileges
```

```
REVOKE PRIVILEGES FROM LUFFY;
```

```
-- drop Role
```

```
DROP Role Admin_role;
```

RESULT:

Thus to Create Databases using DCL and write simple queries to access completed successfully.